

Ödev 18: Sayamayan Bir Sayaç

Aynı sayacı dört biçimde kurun, birinin başarısız olmasını izleyin, sonra iş parçacığı güvenli bir koleksiyonun sizi **korumadığı** bir yarışı düzeltin.

Programınız her seferinde temiz çıkmalı. Bitmeyebilecek her iş parçacığı daemon olmalı, ve her `join` bir zaman aşımına sahip olmalı.

Birinci Bölüm: Dört Sayaç

`BankAccount.java` dosyasını klasik biçimde oluşturun.

`increment()`, `value()` ve `name()` metotlarına sahip bir `Counter` arayüzü ve dört uygulama tanımlayın:

Ad	Mekanizma
<code>UnsafeCounter</code>	düz bir <code>long count</code> ve <code>count++</code>
<code>SynchronizedCounter</code>	bir <code>synchronized</code> bloğu
<code>AtomicCounter</code>	<code>AtomicLong</code>
<code>AdderCounter</code>	<code>LongAdder</code>

Her birini aynı iş yükünden geçirin: 4 iş parçacığı, her biri 100.000 artış. Sonucu, doğru olup olmadığını ve ne kadar sürdüğünü yazdırın.

Senkronize sürüm için tek gereksinim: `this` üzerine değil, bir `private final Object` üzerine kilitleyin. `this` ile neyin ters gidebileceğini söyleyen bir yorum yazın, ve kilitlenmesi kötü olan iki şey daha isimlendirip nedenini söyleyin.

Açıklayın

Kendi kelimelerinizle bir yorumda:

- `count++` gerçekte neye derleniyor
- Tam olarak bir artış kaybeden, iki iş parçacığına ait somut bir araya girme
- Kaybedilen sayının neden **her çalıştırmada değiştiği**, ve bunun sizin için neden tutarlı biçimde yanlış bir cevaptan daha kötü olduğu

İkinci Bölüm: Hiç Bitmeyen Bir Döngü

İki alan tutan bir sınıf yazın: düz bir `boolean` ve bir `volatile boolean`.

Her biri için:

1. Bayrak true olana kadar dönen bir okuyucu iş parçacığı başlatın
2. Kısa süre uyuyup `main` içinden bayrağı ayarlayın
3. Zaman aşımıyla `join` yapın, sonra okuyucunun hâlâ canlı olup olmadığını yazdırın

Düz olan hâlâ canlı olacak. Volatile olan olmayacak.

Açıklayın

Doğru anlaşılmaya değer kısım bu. Şunları kapsayan bir yorum yazın:

- **Bunun neden bir yarış olmadığı.** Kaç iş parçacığı yazıyor, ve kaç kez?
- Derleyicinin o döngüye ne yapmasına izin verildiği, ve neden izin verildiği
- `volatile` neyi değiştiriyor
- `volatile` **birinci bölümün sayacını neden düzeltmezdi.** İki ayrı problemi isimlendirin ve `volatile` hangisini çözüyor söyleyin

Üçüncü Bölüm: Denetle Sonra Davran

Bir `ConcurrentHashMap` kullanın. 8 iş parçacığının hepsi aynı anahtarı almaya çalışsın:

```
if (!map.containsKey("winner")) {
    map.put("winner", id);
    winners.add(id);
}
```

Kaç iş parçacığının kazandığına inandığını yazdırın. Birden fazla olacak.

Sonra tam olarak birinin kazanabileceği şekilde düzeltin ve onu da yazdırın.

Sayacınızın tepe değeri kendisi de yarışsız olmalı. Aynı anda tepeyi yükselten iki iş parçacığı bir güncellemeyi kaybetmemeli, ve bunu nasıl yapacağınızı çözmek alıştırmamızın bir parçası.

Asıl Önemli Soru

Harita bir `ConcurrentHashMap`, yani iş parçacığı güvenli. Bunun yarışı neden **önlemediğini**, iş parçacığı güvenli bir koleksiyonun neyi garanti edip neyi etmediğini, ve harita zaten güvenliyse `putIfAbsent`, `compute` ve `merge` metotlarının neden var olduğunu açıklayan bir yorum yazın.

Yarışın gerçekten tetiklenmesi için ipucu: her iş parçacığı bir sonraki başlamadan önce başlayıp bitiyorsa hiçbir şey çıkmaz. İki `CountDownLatch` nesnesi, her iş parçacığını kapıda tutup hepsini birlikte salmanızı sağlar.

Dördüncü Bölüm: Kimsenin Duymadığı Başarısızlık

Bir `ExecutorService` içine fırlatan bir görev gönderin ve `Future` nesnesini asla incelemeyin. Hiçbir şeyin raporlanmadığını gösterin.

Sonra aynı görevi gönderip `get()` çağırın ve geleni yakalayın. Sebebi yazdırın.

Bir başarısızlığın asla kaybolmamasını sağlamanın üç yolunu sıralayın.

Kabul Kriterleri

- ☐ `java BankAccount.java` ile çalışıyor ve her seferinde temiz çıkıyor
- ☐ Dört sayacın hepsi aynı iş yükünü zamanlamalarıyla çalıştırıyor
- ☐ Güvensiz olan görünür biçimde yanlış; diğer üçü tam olarak doğru
- ☐ `synchronized` bir private final nesne üzerine kilitleniyor, nedenini söyleyen yorumla
- ☐ Kilitlenmesi kötü iki hedef daha, gerekçeleriyle isimlendirilmiş
- ☐ Açıklama kaybettiren somut bir araya girme veriyor
- ☐ Düz bayrak okuyucusu join sonrası hâlâ canlı; volatile olan değil
- ☐ Okuyucu iş parçacıkları daemon ve join'ler zaman aşımli
- ☐ Açıklama bunun neden yarış olmadığını ve `volatile` metodunun birinci bölüm için neden yetmediğini söylüyor
- ☐ Bozuk denetle-sonra-davran birden fazla kazanan bildiriyor
- ☐ Düzeltilmiş sürüm tam olarak bir tane bildiriyor
- ☐ Bir yorum `ConcurrentHashMap` neyi garanti edip etmediğini açıklıyor
- ☐ Sessiz başarısızlık gösteriliyor ve sonra açığa çıkarılıyor

İleri Seviye

Düzeltilmiş denetle-sonra-davran kodunu `compute` ile değiştirin, böylece kazanan iş parçacığı kimliğinin yanına atomik olarak bir zaman damgası da kaydedebilsin.

Sonra lambda'nız çalışırken `compute` metodunun neyi tuttuğunu, ve o lambda'nın içinde başka bir haritanın metodunu çağırmanın, IO yapmanın ya da ikinci bir kilit almanın neden kötü fikir olacağını bir yorumda yazın.

İpucu, güvensiz sayaç doğru çıkıyorsa

İş parçacıklarının gerçekten çakıştığından emin olun. İş yükünüz yeterince küçükse ya da havuz tek elemanlıysa, her görev bir sonraki başlamadan biter ve araya girecek hiçbir şey olmaz. Daha fazla yineleme ve çekirdek sayısı kadar iş parçacığı bunu sağlar.